Intermediate + Advanced SQL PLP Assignment

Q1) Topic: Window Functions: ROW_NUMBER, RANK, LAG, LEAD & Frame Clauses

Using orders (id, customer_id, order_date, total_amount):

a) ROW_NUMBER() to assign a sequential number to each customer's orders by order_date (row 1 = first order ever).

Query | Query History

```
1  SELECT ROW_NUMBER() OVER (
2      PARTITION BY customer_id
3      ORDER BY order_date) AS row_number,id,customer_id,order_date,total_amount
4  FROM orders;
```

Data Output | Messages | Notifications

Showing rows: 1 to 16 | Page No: 1 of 1

	row_number bigint	id [PK] integer	customer_id integer	order_date date	total_amount numeric (10,2)
1	1	1	1	2024-01-10	4000.00
2	2	2	1	2024-02-15	3500.00
3	3	3	1	2024-03-20	3000.00
4	4	15	1	2025-01-14	6000.00
5	1	4	2	2024-01-25	2500.00
6	2	5	2	2024-04-12	1800.00
7	1	6	3	2024-01-05	6000.00
8	2	7	3	2024-02-18	5500.00
9	3	8	3	2024-02-25	2000.00
10	4	16	3	2025-02-11	3000.00
11	1	9	4	2024-03-08	4500.00

b) RANK() and DENSE_RANK() together to rank all orders by total_amount descending. Show a row where they differ.

Query | Query History

```
SELECT RANK() OVER(ORDER BY total_amount DESC) as rank,
DENSE_RANK() OVER (ORDER BY total_amount DESC) as dense_rank,id,customer_id,order_date,total_amount
FROM orders;
```

Data Output | Messages | Notifications

Showing rows: 1 to 16 | Page No: 1 of 1

	rank bigint	dense_rank bigint	id [PK] integer	customer_id integer	order_date date	total_amount numeric (10,2)
1	1	1	11	5	2024-01-12	7000.00
2	2	2	6	3	2024-01-05	6000.00
3	2	2	15	1	2025-01-14	6000.00
4	4	3	7	3	2024-02-18	5500.00
5	5	4	9	4	2024-03-08	4500.00
6	6	5	1	1	2024-01-10	4000.00
7	6	5	12	5	2024-06-21	4000.00
8	8	6	2	1	2024-02-15	3500.00
9	9	7	16	3	2025-02-11	3000.00
10	9	7	3	1	2024-03-20	3000.00
11	11	8	4	2	2024-01-25	2500.00
12	12	9	8	3	2024-02-25	2000.00

Total rows: 16 | Query complete 00:00:00.139 | CRLF | Ln 2, Col 62

Observe the 2nd, 3rd and 4th row from the output:

1. `RANK()` skips a rank after a tie.
2. `DENSE_RANK()` does not skip ranks after a tie.

c) LAG() for prev_order_amount and LEAD() for next_order_amount per customer. Show NULL where no previous/next.

Query | Query History

```
SELECT id,customer_id,order_date,total_amount,
LAG(total_amount) OVER(PARTITION BY customer_id ORDER BY order_date) as prev_order_amount,
LEAD(total_amount) OVER(PARTITION BY customer_id ORDER BY order_date) as next_order_amount
FROM orders;
```

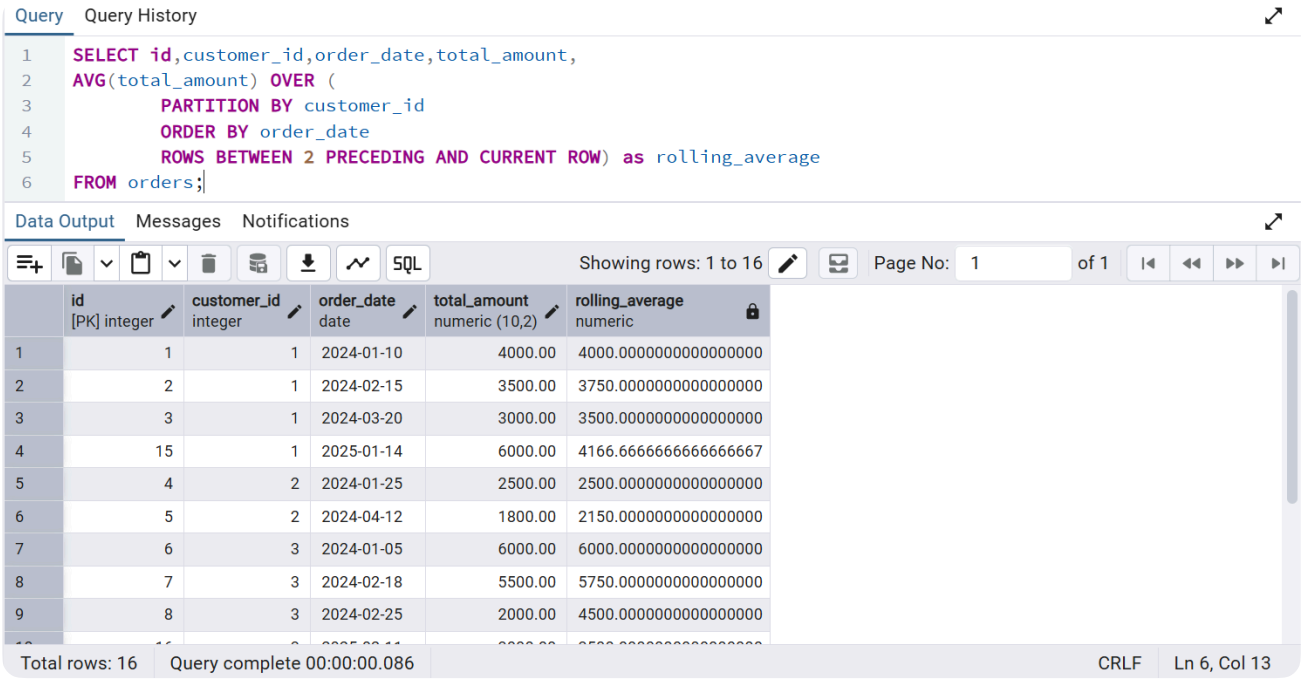
Data Output | Messages | Notifications

Showing rows: 1 to 16 | Page No: 1 of 1

	id [PK] integer	customer_id integer	order_date date	total_amount numeric (10,2)	prev_order_amount numeric	next_order_amount numeric
1	1	1	2024-01-10	4000.00	[null]	3500.00
2	2	1	2024-02-15	3500.00	4000.00	3000.00
3	3	1	2024-03-20	3000.00	3500.00	6000.00
4	15	1	2025-01-14	6000.00	3000.00	[null]
5	4	2	2024-01-25	2500.00	[null]	1800.00
6	5	2	2024-04-12	1800.00	2500.00	[null]
7	6	3	2024-01-05	6000.00	[null]	5500.00
8	7	3	2024-02-18	5500.00	6000.00	2000.00
9	8	3	2024-02-25	2000.00	5500.00	3000.00
10	16	3	2025-02-11	3000.00	2000.00	[null]
11	9	4	2024-03-08	4500.00	[null]	1200.00

Total rows: 16 | Query complete 00:00:00.104 | CRLF | Ln 2, Col 68

d) 3-order rolling average of total_amount per customer using SUM() OVER() with ROWS BETWEEN 2 PRECEDING AND CURRENT ROW.



e) Predict the result set: -- orders: (1,1,'2024-01-01',100), (2,1,'2024-02-01',300),(3,2,'2024-01-01',400)

```
SELECT customer_id, total_amount,
RANK() OVER(ORDER BY total_amount DESC) AS rnk,
LAG(total_amount) OVER(PARTITION BY customer_id ORDER BY order_date) AS
prev
FROM orders;
```

Final result set:

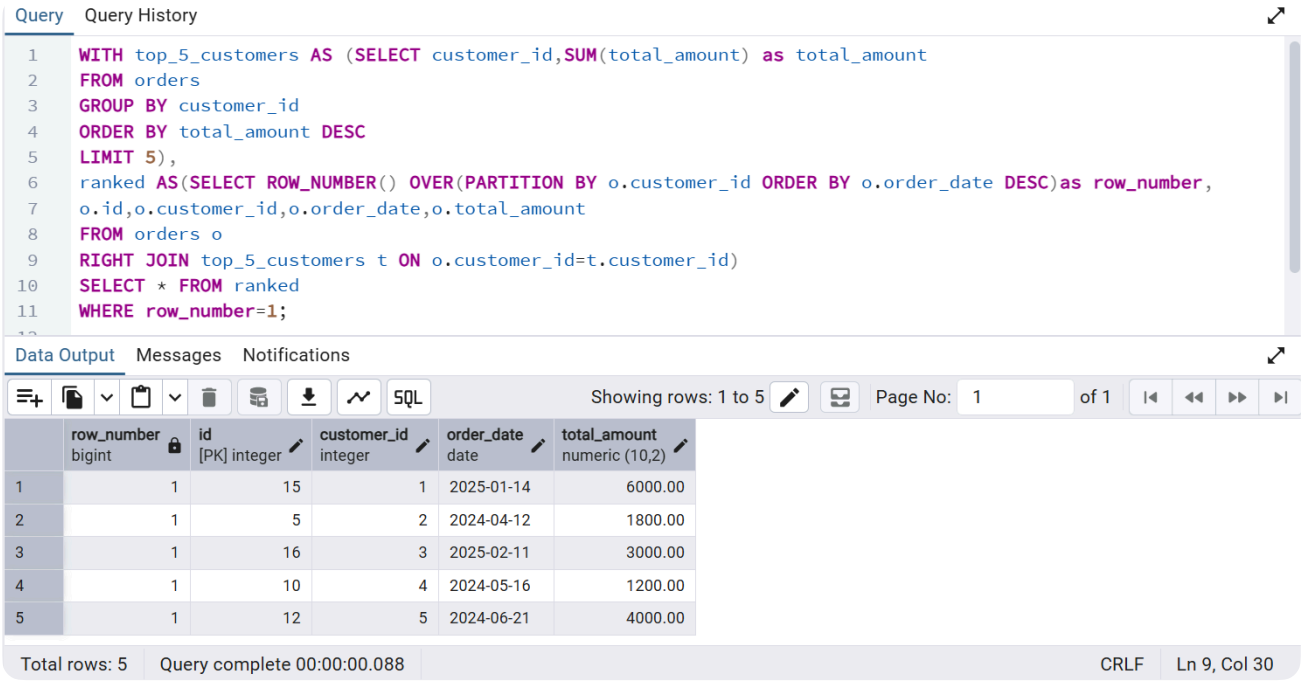
id	customer_id	order_date	total_amount	rnk	prev
1	1	2024-01-01	100	3	`NULL`
2	1	2024-02-01	300	2	100
3	2	2024-01-01	400	1	`NULL`

1. `RANK()` assigns ranks based on `total_amount` in descending order.
2. `LAG()` returns the previous order amount within the same customer's partition.
3. Customer 2 has no previous order in their partition, so `prev` is `NULL`.

Q2) Topic: CTEs, Recursive CTEs, UNION / INTERSECT / EXCEPT & ROLLUP / CUBE / GROUPING SETS

Using the same e-commerce tables:

a) CTE named top_customers: top 5 customers by total spending. Join the CTE with orders to show their most recent purchase.



```
WITH top_5_customers AS (SELECT customer_id,SUM(total_amount) as total_amount
FROM orders
GROUP BY customer_id
ORDER BY total_amount DESC
LIMIT 5),
ranked AS(SELECT ROW_NUMBER() OVER(PARTITION BY o.customer_id ORDER BY o.order_date DESC)as row_number,
o.id,o.customer_id,o.order_date,o.total_amount
FROM orders o
RIGHT JOIN top_5_customers t ON o.customer_id=t.customer_id)
SELECT * FROM ranked
WHERE row_number=1;
```

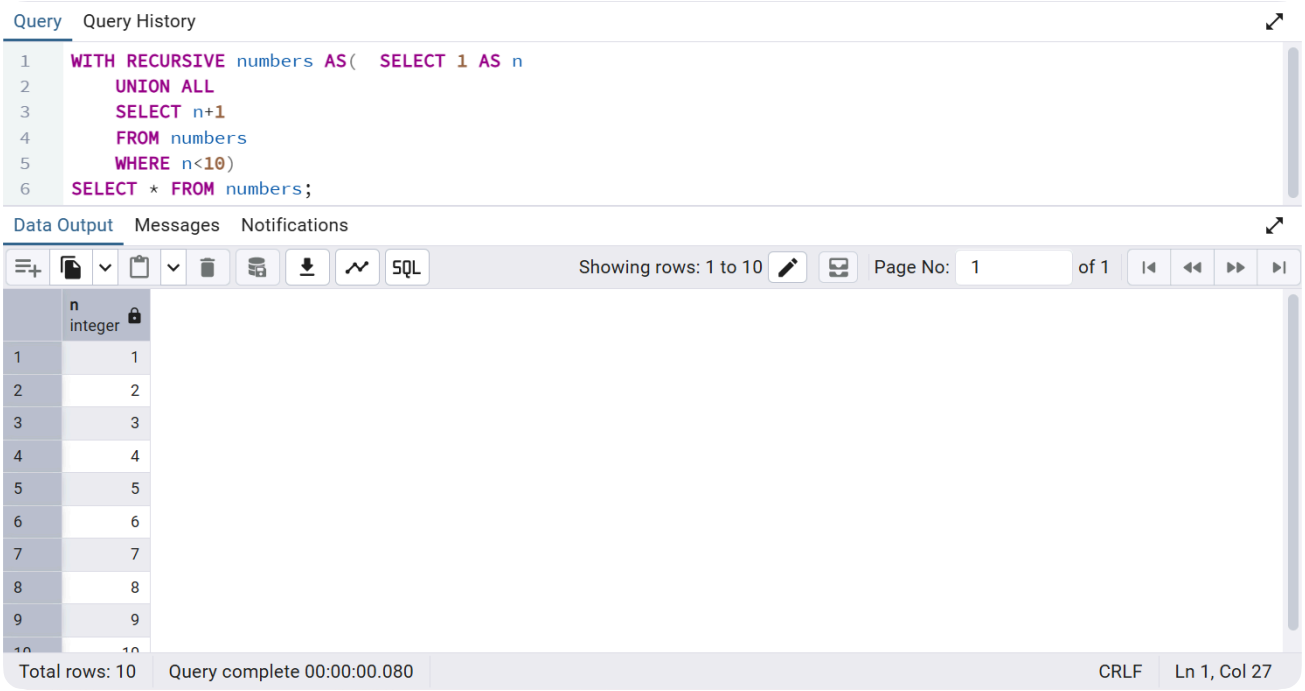
Data Output | Messages | Notifications

Showing rows: 1 to 5 | Page No: 1 of 1

	row_number bigint	id [PK] integer	customer_id integer	order_date date	total_amount numeric (10,2)
1	1	15	1	2025-01-14	6000.00
2	1	5	2	2024-04-12	1800.00
3	1	16	3	2025-02-11	3000.00
4	1	10	4	2024-05-16	1200.00
5	1	12	5	2024-06-21	4000.00

Total rows: 5 | Query complete 00:00:00.088 | CRLF | Ln 9, Col 30

b) Recursive CTE generating numbers 1 to 10. Explain when recursive CTEs are useful (e.g., hierarchical data, org trees).



```
WITH RECURSIVE numbers AS(  SELECT 1 AS n
    UNION ALL
    SELECT n+1
    FROM numbers
    WHERE n<10)
SELECT * FROM numbers;
```

Data Output | Messages | Notifications

Showing rows: 1 to 10 | Page No: 1 of 1

	n integer
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9

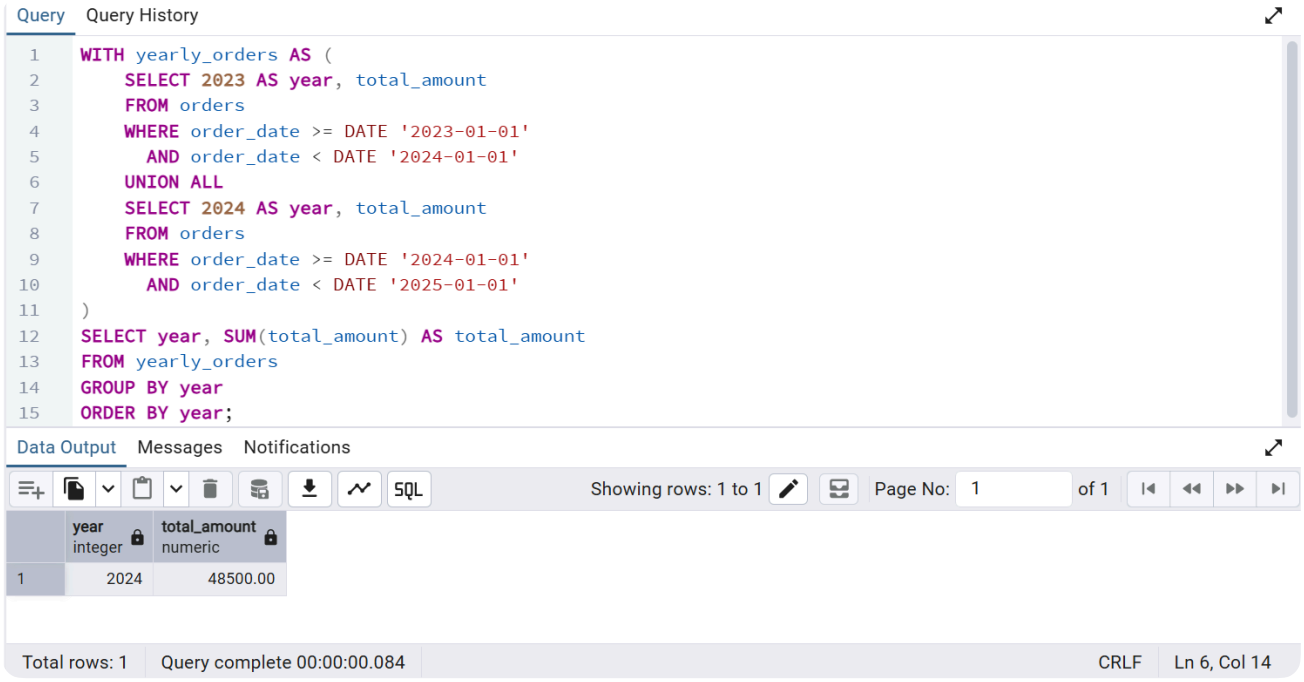
Total rows: 10 | Query complete 00:00:00.080 | CRLF | Ln 1, Col 27

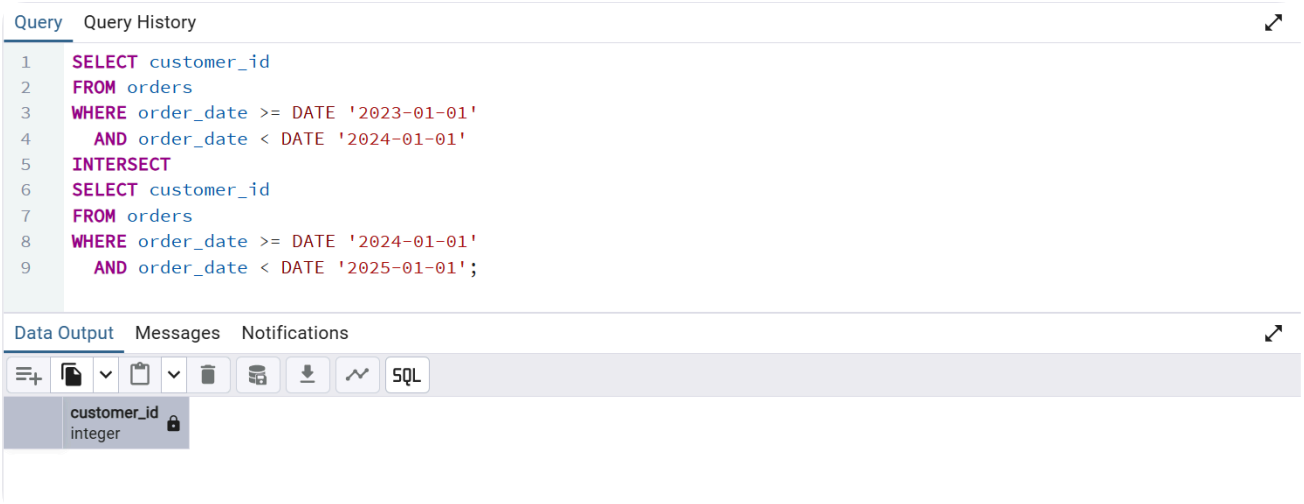
Recursive CTEs are useful when data or calculations involve repeated steps or hierarchical relationships:

1. **Hierarchical data:** Retrieve categories and their subcategories.

2. **Organizational trees:** Find all employees reporting directly or indirectly to a manager.
3. **Bill of materials**: Find all components and subcomponents required to assemble a product.
4. **Graph traversal:** Explore connected nodes or relationships.
5. **Sequence generation:** Generate numbers, dates, or other sequential values.

c) UNION ALL combining 2023 and 2024 orders with a year column. Wrap in CTE, GROUP BY year to compare totals. Use INTERSECT to find customer_ids ordering in BOTH years.





d) ROLLUP on (category, city) showing revenue at: individual, category subtotal, and grand total. Rewrite using GROUPING SETS for only 2 of the 3 levels.

With `ROLLUP`:

Query | Query History

```
SELECT p.category, c.city, SUM(oi.quantity * p.price) AS revenue
FROM orders AS o
JOIN customers AS c ON o.customer_id = c.id
JOIN order_items AS oi ON o.id = oi.order_id
JOIN products AS p ON oi.product_id = p.id
GROUP BY ROLLUP (p.category, c.city)
ORDER BY p.category, c.city;
```

Data Output | Messages | Notifications

Showing rows: 1 to 6 | Page No: 1 of 1

	category character varying (50)	city character varying (100)	revenue numeric
1	Electronics	Delhi	3500.00
2	Electronics	Mumbai	4998.00
3	Electronics	[null]	8498.00
4	Stationery	Mumbai	490.00
5	Stationery	[null]	490.00
6	[null]	[null]	8988.00

Total rows: 6 | Query complete 00:00:00.124 | CRLF | Ln 5, Col 19

With `GROUPING SETS`:

Query | Query History

```
SELECT p.category, c.city, SUM(oi.quantity * p.price) AS revenue
FROM orders AS o
JOIN customers AS c ON o.customer_id = c.id
JOIN order_items AS oi ON o.id = oi.order_id
JOIN products AS p ON oi.product_id = p.id
GROUP BY GROUPING SETS ((p.category, c.city),())
ORDER BY p.category, c.city;
```

Data Output | Messages | Notifications

Showing rows: 1 to 4 | Page No: 1 of 1

	category character varying (50)	city character varying (100)	revenue numeric
1	Electronics	Delhi	3500.00
2	Electronics	Mumbai	4998.00
3	Stationery	Mumbai	490.00
4	[null]	[null]	8988.00

e) Predict the result set: -- sales(category,city,amount): ('A','X',100),('A','Y',200),('B','X',300)

```
SELECT category, city, SUM(amount)
FROM sales
GROUP BY ROLLUP(category,city);
```

Final result set:

category	city	SUM(amount)
A	X	100
A	Y	200
A	`NULL`	300

category	city	SUM(amount)
B	X	300
B	`NULL`	300
`NULL`	`NULL`	600

Explanation:

1. `(category, city)` gives individual group totals.
2. `(category)` gives category subtotals.
3. `()` gives the grand total of all sales.

Q3) Topic: Query Optimization: EXPLAIN ANALYZE, Indexes & Materialized Views

Given:

```
SELECT *
FROM orders o
JOIN customers c ON o.customer_id=c.id
WHERE c.city='Mumbai' AND o.order_date>'2024-01-01'
ORDER BY o.total_amount DESC;
```

a) Write the EXPLAIN ANALYZE version. Describe what to look for: Seq Scan vs Index Scan, actual rows, cost, buffers.

Query | Query History

```
1  EXPLAIN ANALYZE
2  SELECT * FROM orders AS o
3  JOIN customers AS c
4  ON o.customer_id = c.id
5  WHERE c.city = 'Mumbai'AND o.order_date > DATE '2024-01-01'
6  ORDER BY o.total_amount DESC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 14 | Page No: 1 of 1

	QUERY PLAN text
1	Sort (cost=40.51..40.52 rows=6 width=988) (actual time=0.110..0.112 rows=6 loops=1)
2	Sort Key: o.total_amount DESC
3	Sort Method: quicksort Memory: 25kB
4	-> Hash Join (cost=11.01..40.43 rows=6 width=988) (actual time=0.055..0.062 rows=6 loops=1)
5	Hash Cond: (o.customer_id = c.id)
6	-> Seq Scan on orders o (cost=0.00..28.12 rows=483 width=28) (actual time=0.027..0.030 rows=16 loops=1)
7	Filter: (order_date > '2024-01-01'::date)
8	-> Hash (cost=11.00..11.00 rows=1 width=960) (actual time=0.017..0.017 rows=2 loops=1)
9	Buckets: 1024 Batches: 1 Memory Usage: 9kB
10	-> Seq Scan on customers c (cost=0.00..11.00 rows=1 width=960) (actual time=0.011..0.013 rows=2 loo...
11	Filter: ((city)::text = 'Mumbai'::text)
12	Rows Removed by Filter: 5
13	Planning Time: 0.286 ms
14	Execution Time: 0.148 ms

What to look for in the execution plan:

1. **Seq Scan:** PostgreSQL scans the table sequentially. This can be efficient for small tables or queries returning many rows.
2. **Actual rows**: The actual number of rows processed or returned by each plan node.
3. **Cost:** Estimated startup and total costs, expressed in PostgreSQL's internal cost units, not milliseconds.
4. **Buffers:** Use `EXPLAIN (ANALYZE, BUFFERS)` to inspect shared-buffer hits and reads.

b) Propose and write CREATE INDEX statements. Explain why a composite index on (customer_id, order_date) is better than two single-column indexes.

Query | Query History

```
1  CREATE INDEX idx_customers_city
2  ON customers(city);
3
4  CREATE INDEX idx_orders_customer_date
5  ON orders(customer_id, order_date);
```

Data Output | Messages | Notifications

```
CREATE INDEX

Query returned successfully in 85 msec.
```

Why use a composite index on `(customer_id, order_date)`?

1. The query joins `orders.customer_id` to `customers.id`.
2. It filters orders by `order_date`.
3. A composite index can support lookups using `customer_id` and filtering by `order_date` within that customer.
4. Two separate indexes may require PostgreSQL to combine index results or choose just one index. The composite index can be more efficient when both columns are used together.
   The best index depends on table size, data distribution, selectivity, and the execution plan. Because the query filters `customers.city` first logically, PostgreSQL may choose different join orders based on its estimates.

c) Rewrite SELECT *to select only needed columns. Explain why SELECT* is harmful on wide tables.

Query | Query History

```
1  SELECT c.name, c.city, o.order_date, o.total_amount
2  FROM orders AS o
3  JOIN customers AS c ON o.customer_id = c.id
4  WHERE c.city = 'Mumbai' AND o.order_date > DATE '2024-01-01'
5  ORDER BY o.total_amount DESC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 6 | Page No: 1 of 1

	name character varying (100)	city character varying (100)	order_date date	total_amount numeric (10,2)
1	Aarav	Mumbai	2025-01-14	6000.00
2	Aarav	Mumbai	2024-01-10	4000.00
3	Aarav	Mumbai	2024-02-15	3500.00
4	Aarav	Mumbai	2024-03-20	3000.00
5	Diya	Mumbai	2024-01-25	2500.00
6	Diya	Mumbai	2024-04-12	1800.00

Why avoid `SELECT *`?

1. Retrieves unnecessary columns, increasing data transfer and memory usage.
2. Can increase disk I/O, particularly for wide tables.
3. Makes queries more vulnerable to schema changes.
4. Can prevent index-only scans when the selected columns aren't covered by an index.

d) Create a materialized view mv_customer_revenue computing total revenue per customer. Write the REFRESH MATERIALIZED VIEW command and explain when to use it vs regular views.

Query | Query History

```
1  CREATE MATERIALIZED VIEW mv_customer_revenue AS
2  SELECT c.id AS customer_id, c.name AS customer_name,COALESCE(SUM(o.total_amount), 0) AS total_revenue
3  FROM customers AS c
4  LEFT JOIN orders AS o ON c.id = o.customer_id
5  GROUP BY c.id, c.name;
```

Data Output | Messages | Notifications

```
SELECT 7

Query returned successfully in 126 msec.
```

Query | Query History

```
1  REFRESH MATERIALIZED VIEW mv_customer_revenue;
```

Data Output | Messages | Notifications

```
REFRESH MATERIALIZED VIEW

Query returned successfully in 74 msec.
```

Regular view	Materialized view
Stores the query definition	Stores the query result
Usually calculates results when queried	Reads previously computed results
Reflects current underlying data when queried	Can become stale until refreshed
Doesn't store the result set itself	Requires storage for the result set

Materialized view is used when an expensive aggregation is queried frequently and slightly stale results are acceptable.

A regular view is preferable when results need to reflect current data and the query cost is acceptable.

e) Predict what changes:

```
-- Before index:EXPLAIN shows Seq Scan, cost=0.00..450.00, rows=10000

CREATE INDEX idx_ord ON orders(customer_id, order_date);

-- After index: EXPLAIN shows ______, cost=0.00..___?
```

1. **Scan type:** It may change from `Seq Scan` to `Index Scan` or `Bitmap Index Scan`
2. **Cost:** It may decrease from `450.00` to a lower estimated cost.

The index on `(customer_id, order_date)` is most useful when the query can use `customer_id`, or both `customer_id` and `order_date`, as index conditions. It does not directly provide rows sorted by `total_amount`.

> **Important**
>
> PostgreSQL may still choose a sequential scan if many rows match, the table is small, or using the index is estimated to be more expensive.

Q4) Topic: ACID Transactions & Isolation Levels

Bank transfer (accounts: id, balance; transactions: id, from_account, to_account, amount, created_at):

a) Transaction: deduct 5000 from account 101, credit 5000 to account 202, insert a transactions record. Rollback if balance < 0.

Query | Query History

```
1  BEGIN;
2  UPDATE accounts
3  SET balance = balance - 5000
4  WHERE id = 101;
5  UPDATE accounts
6  SET balance = balance + 5000
7  WHERE id = 202;
8  INSERT INTO transactions (from_account, to_account, amount)
9  VALUES (101, 202, 5000);
10 COMMIT;
```

Data Output | Messages | Notifications

```
COMMIT

Query returned successfully in 244 msec.
```

b) Add a check so insufficient funds triggers rollback and error message.

Query | Query History

```
BEGIN;
DO $$
DECLARE current_balance NUMERIC(12, 2);
BEGIN
    SELECT balance INTO current_balance
    FROM accounts
    WHERE id = 101
    FOR UPDATE;
    IF NOT FOUND THEN RAISE EXCEPTION 'Source account does not exist';
    END IF;
    IF NOT EXISTS (SELECT 1 FROM accounts WHERE id = 202)
    THEN RAISE EXCEPTION 'Destination account does not exist';
    END IF;
    IF current_balance < 5000
    THEN RAISE EXCEPTION 'Insufficient funds! Available balance: %', current_balance;
    END IF;
    UPDATE accounts SET balance = balance - 5000
    WHERE id = 101;
    UPDATE accounts SET balance = balance + 5000
    WHERE id = 202;
    INSERT INTO transactions (from_account, to_account, amount) VALUES (101, 202, 5000);
END $$;
COMMIT;
```

Data Output | Messages | Notifications

```
COMMIT

Query returned successfully in 67 msec.
```

c) Explain 'dirty read', 'non-repeatable read', and 'phantom read'. Which isolation level prevents each?

These are **concurrency problems** that can occur when multiple transactions access or modify the same database simultaneously.

1. Dirty Read

A dirty read occurs when a transaction reads data that has been modified by another transaction but not yet committed.

Example

- Transaction A updates an account balance from ₹5,000 to ₹3,000 but does not >commit.
- Transaction B reads the balance as ₹3,000.
- Transaction A rolls back, restoring the balance to ₹5,000.

Transaction B has read data that was never committed.
Prevented by: `READ COMMITTED` and higher isolation levels.

2. Non-repeatable Read

A non-repeatable read occurs when a transaction reads the same row twice and gets different values because another transaction modified and committed that row in

between.

Example

- Transaction A reads an account balance of ₹5,000.
- Transaction B updates the balance to ₹7,000 and commits.
- Transaction A reads the same row again and gets ₹7,000.

The same query returns a different value within Transaction A.
Prevented by: `REPEATABLE READ` and `SERIALIZABLE`.

3. Phantom Read

A phantom read occurs when a transaction executes the same query twice and gets a different set of rows because another transaction inserted, deleted, or modified rows matching the query condition.

Example

- Transaction A queries all accounts with a balance greater than ₹5,000 and gets 3 rows.
- Transaction B inserts another account with a balance of ₹8,000 and commits.
- Transaction A executes the same query again and gets 4 rows.

The additional row is called a phantom row.
Prevented by: `SERIALIZABLE` under the SQL standard. PostgreSQL's `REPEATABLE READ` also prevents phantom reads.

d) Explain optimistic vs pessimistic locking. Give a real-world scenario where each is appropriate.

Locking strategies control how multiple transactions access and modify the same data to prevent conflicts and maintain data consistency.

1. Optimistic Locking

Optimistic locking assumes that conflicts between transactions are rare. It allows multiple transactions to access the same data without locking it initially.

Before updating, the transaction checks whether the data has changed since it was read. If another transaction has modified it, the update is rejected.

> **Example**
>
> Suppose two users are editing the same customer profile.
>
> 1. User A reads a customer's record with version `1`.
> 2. User B also reads the same record with version `1`.
> 3. User A updates the record and increments the version to `2`.
> 4. User B attempts to update using version `1`.
> 5. The update fails because the current version is now `2`.

2. Pessimistic Locking

Pessimistic locking assumes that conflicts are likely to occur. It locks the data before modifying it so that other transactions cannot modify the locked rows until the current transaction finishes.

> **Example**
>
> Suppose two customers attempt to withdraw money from the same bank account >simultaneously.
>
> 1. Transaction A locks the account row.
> 2. Transaction A checks the available balance.
> 3. Transaction B attempts to access the same row for an update and must wait.
> 4. Transaction A updates the balance and commits.
> 5. Transaction B can then proceed using the updated balance.

Real-world scenario: Banking transactions, where simultaneous withdrawals or transfers must not cause incorrect account balances.

Comparison

Feature	Optimistic locking	Pessimistic locking
Assumption	Conflicts are rare	Conflicts are likely
Locking	No lock during initial read	Locks data before modification

Feature	Optimistic locking	Pessimistic locking
Conflict handling	Detects conflicts during update	Makes competing transactions wait
Performance	Suitable for low-conflict workloads	Can reduce concurrency
Risk	Updates may need to be retried	Can cause waiting and deadlocks
Common use	Profile updates, document editing	Banking, inventory management

Key takeaway:

1. Optimistic locking: Check for conflicts before accepting an update.
2. Pessimistic locking: Prevent conflicts by locking data before updating it.

e) Predict the result: -- accounts: (1,5000),(2,5000)

```
BEGIN;
UPDATE accounts SET balance=balance-1000
WHERE id=1;
UPDATE accounts SET balance=balance+1000 WHERE id=2;
COMMIT;
SELECT balance FROM accounts ORDER BY id;
```

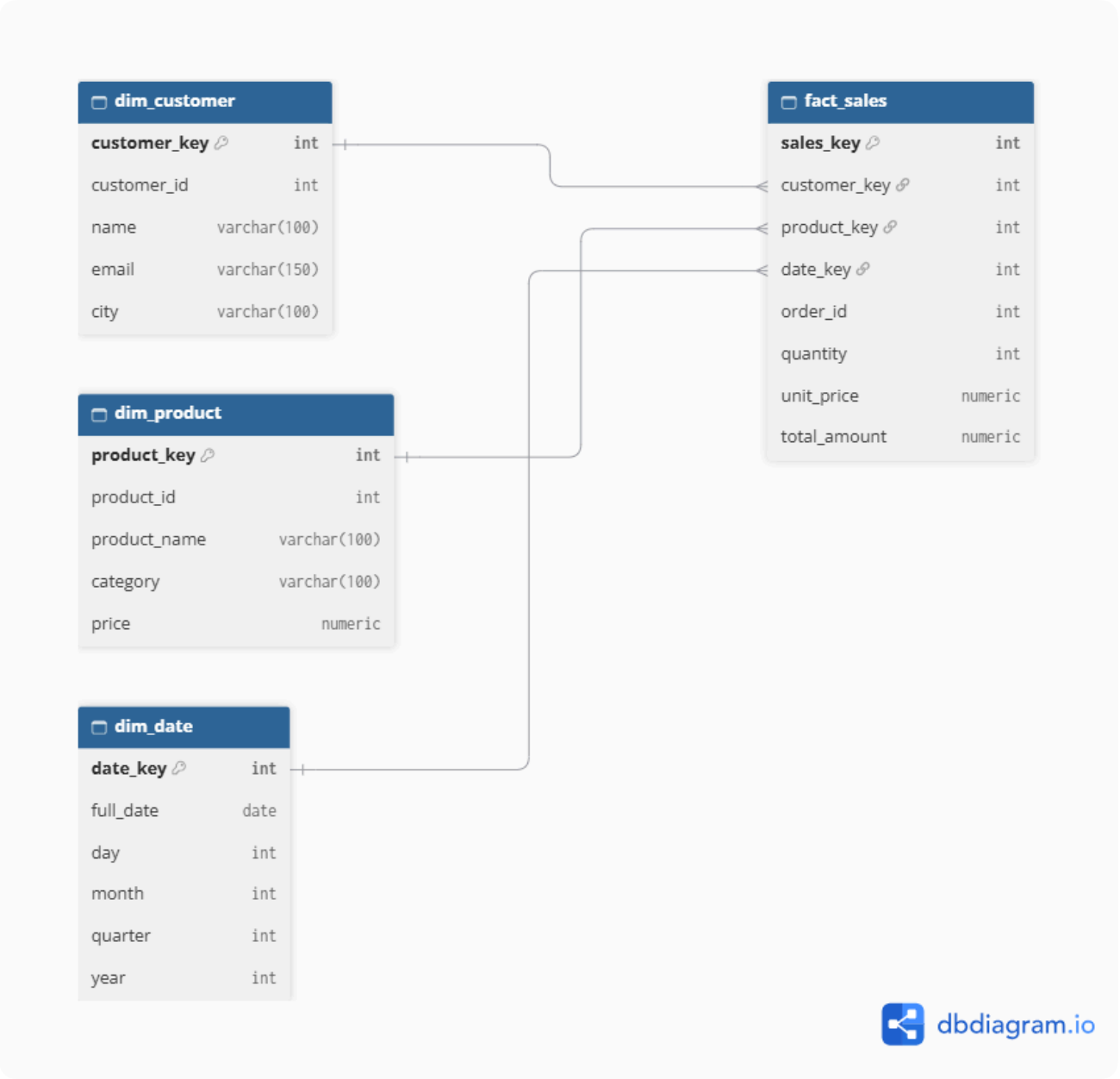
Final result set:

balance
4000
6000

Q5)Topic: Star Schema, Snowflake Schema & SCD Type 2

Designing a data warehouse for the e-commerce platform:

a) Design a star schema: name fact table and at least 3 dimension tables. List columns with PKs and FKs. Explain your choices.



Explanation:

1. `fact_sales` is the central fact table.
2. Dimension tables contain descriptive information.
3. PK and FK relationships connect the tables.
4. One dimension record can be associated with multiple fact records.

b) CREATE TABLE for dim_customer with SCD Type 2 columns: customer_key, customer_id, name, email, city, effective_from, effective_to (nullable), is_current.

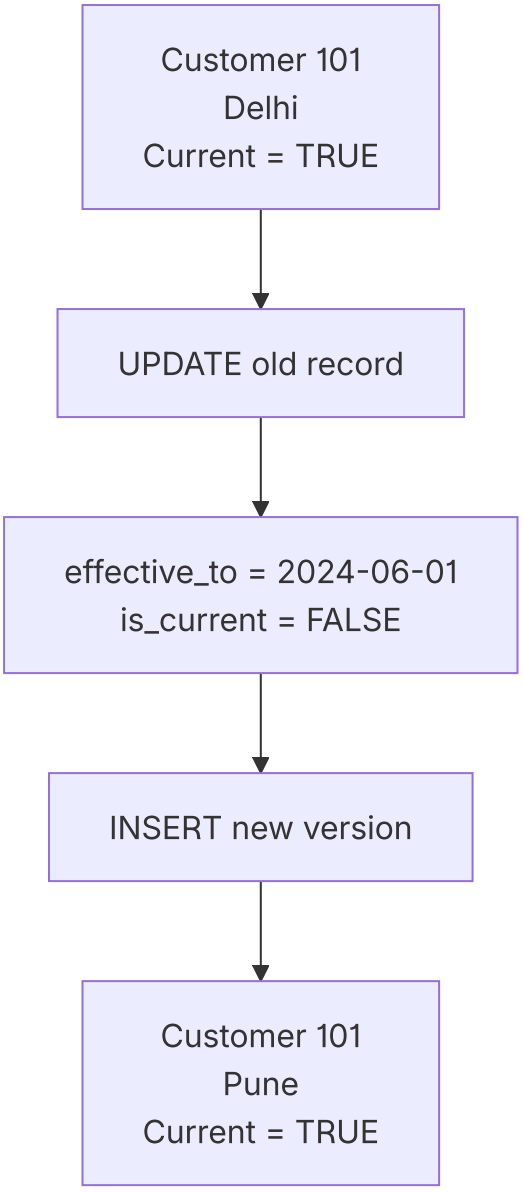


Explanation:

1. `customer_id` remains the same across versions.
2. `customer_key` changes for every new version.
3. `effective_from` and `effective_to` track validity periods.
4. `is_current` identifies the latest version.

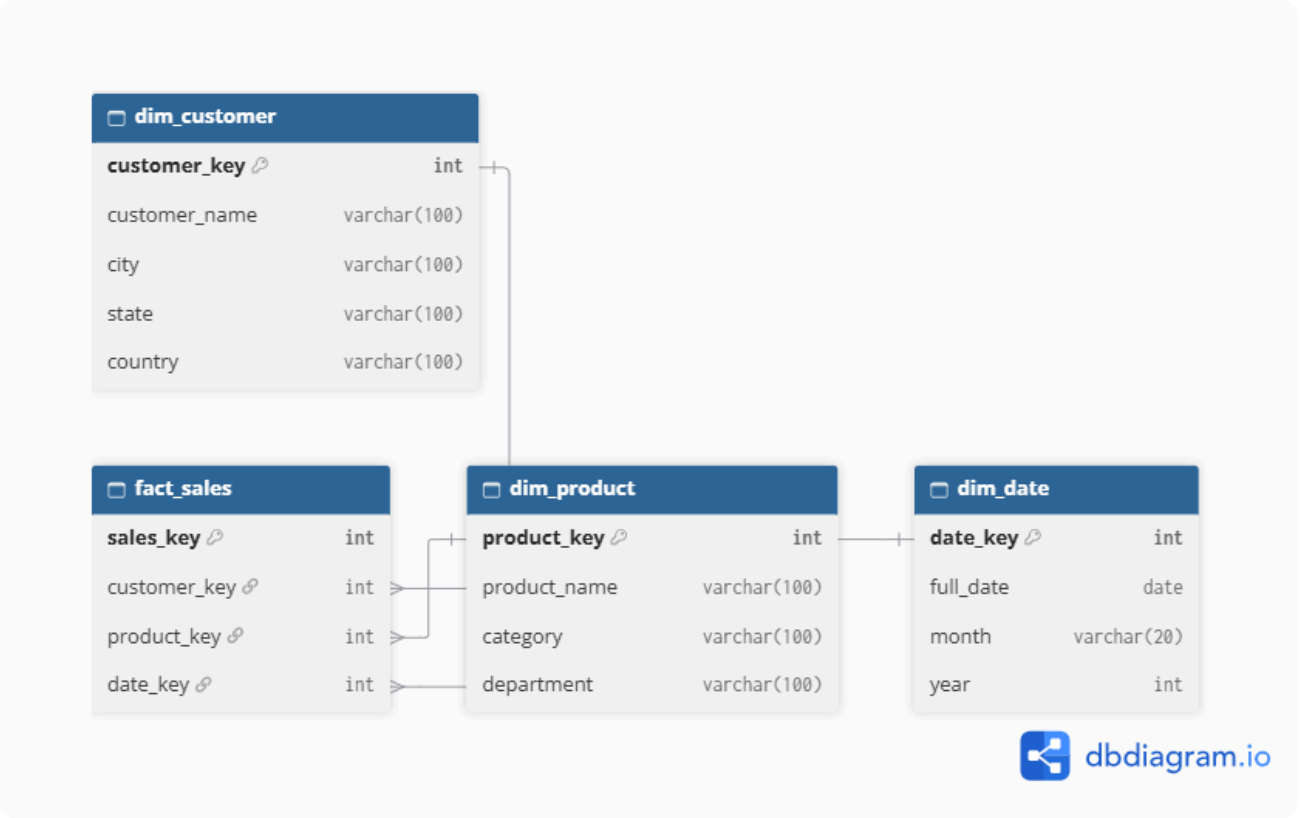
c) SQL to handle customer changing city 'Delhi' to 'Pune': UPDATE to expire old record, INSERT new version with is_current = true.

This is a data-flow representation of the UPDATE and INSERT operations.



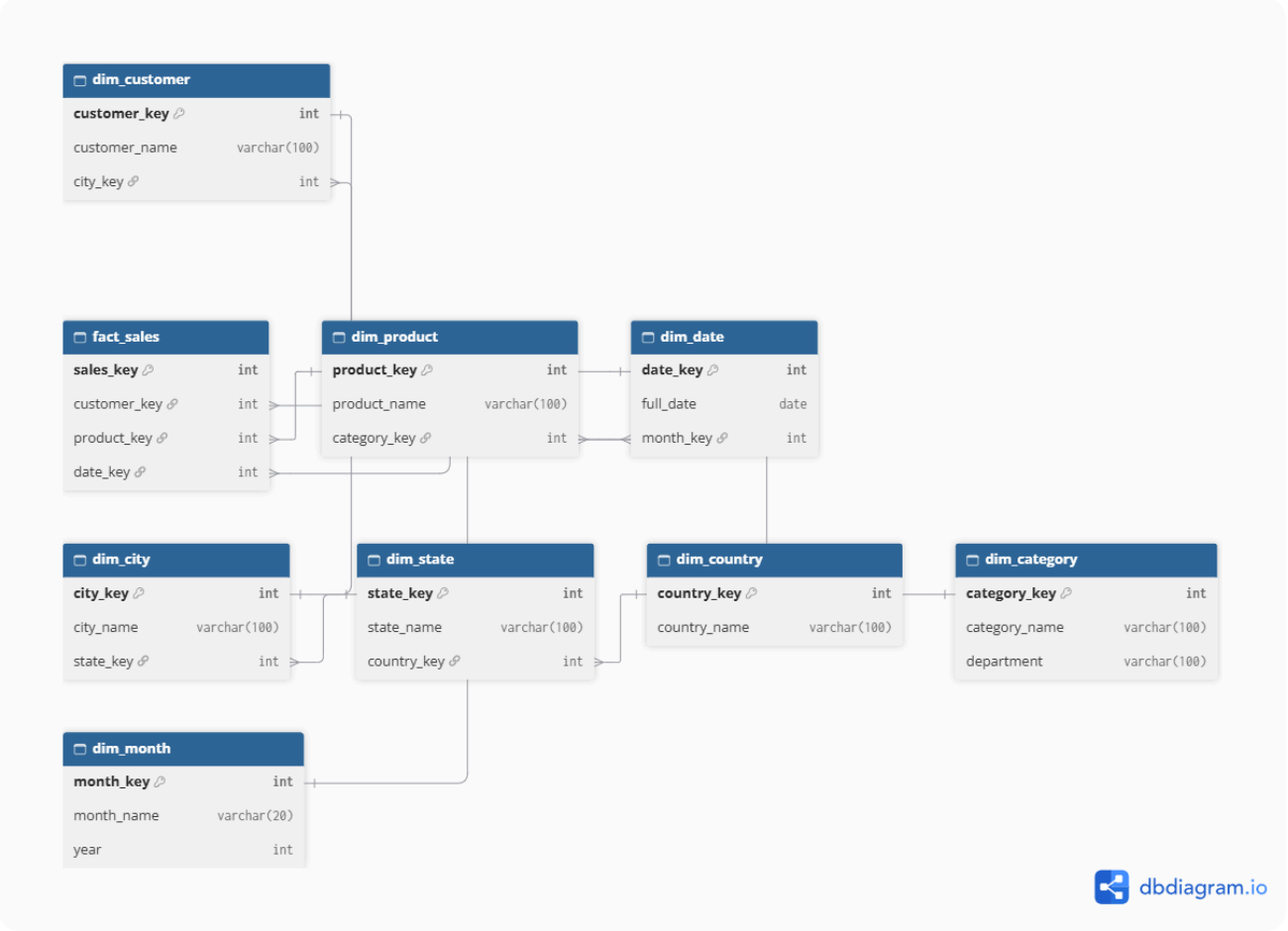
d) Explain how snowflake schema differs from star schema. When would you choose snowflake? Give one concrete trade-off.

Star Schema:



1. A central fact table connects directly to dimension tables.
2. Dimension tables are generally denormalized.
3. Requires fewer joins, making queries simpler and often faster.
4. May contain redundant data.

Snowflake Schema:



1. Dimension tables are normalized into multiple related tables.
2. Reduces data redundancy and improves data consistency.
3. Requires more joins, making queries more complex.

Main difference: In a star schema, customer location details are stored together in `dim_customer`. In a snowflake schema, location details are separated into `dim_city`, `dim_state`, and `dim_country`.

When to choose Snowflake Schema?

A snowflake schema is chosen when dimension tables contain significant redundant data or complex hierarchies that benefit from normalization.

Concrete trade-off:

1. **Advantage:** Reduces data redundancy and improves data consistency.
2. **Disadvantage:** Requires additional joins, which can increase query complexity and affect query performance.

e) Predict the result: -- dim_customer: (key=1,cust_id=101,city='Delhi',is_current=TRUE)

```
UPDATE dim_customer
SET effective_to='2024-06-01', is_current=FALSE WHERE customer_key=1;

INSERT INTO dim_customer VALUES (2,101,'Priya','p@x.com','Pune','2024-06-
01',NULL,TRUE);

SELECT customer_key,city,is_current
FROM dim_customer
WHERE customer_id=101 ORDER BY customer_key;
```

Final query output:

customer_key	city	is_current
1	Delhi	FALSE
2	Pune	TRUE